2η Εργασία

Μπαρμπαγιάννος Βασίλειος
ΑΕΜ: 10685
Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών

Περίληψη—Υλοποίηση Μηχανής Διανυσματικής Υποστήριξης (**Support Vector Machine - SVM**) για την κατηγοριοποίηση πολλών κλάσεων. Σύγκριση της ακρίβειας (**accuracy**) με ένα πολυστρωματικό νευρωνικό δίκτυο (**Multilayer Perceptron - MLP**) που χρησιμοποιεί **Hinge Loss** και τους κατηγοριοποιητές 1-NN, 3-NN και **Nearest Centroid**.

I . Εισαγωγή

Στόχος της εργασίας είναι να υλοποιήσουμε μια Μηχανή Διανυσματικής Υποστήριξης (Support Vector Machine) για την κατηγοριοποίηση πολλών κλάσεων. Επιλέγουμε το dataset CIFAR-100 όπως και στις προηγούμενες εργασίες. Θα εκπαιδεύσουμε το SVM για να κατηγοριοποιεί τις 100 κλάσεις που εμπεριέχονται στο dataset.

Επίσης, η εργασία μάς ζητά την υλοποίηση ενός πολυστρωματικού νευρωνικού δικτύου (Multilayer Perceptron) για την κατηγοριοποίηση των ίδιων 100 κλάσεων του dataset. Για τη βελτιστοποίηση θα χρησιμοποιηθεί Hinge Loss (περισσότερες πληροφορίες στο σύνδεσμο του FreeCodeCamp https://medium.com/@noorfatimaafzalbutt/understanding-perceptron-loss-function-hinge-loss-binary-cross-entropy-and-the-sigmoid-function-bbb42d128ef6). Το MLP αυτό θα έχει ένα κρυφό επίπεδο.

Από τα παραπάνω θα εξαχθεί η ακρίβειά των και θα συγκριθεί με την ακρίβεια που προκύπτει από τους ταξινομητές 1 Nearest Neighbour και 3 Nearest Neighbours καθώς και Nearest Centroid.

II . Προετοιμασία των δεδομένων

Για την προετοιμασία των δεδομένων χρησιμοποίησα τον StandardScaler της βιβλιοθήκης sklearn.preprocessing για να κανονικοποιήσω τα δεδομένα ώστε να έχουν μέση τιμή μηδέν και τυπική απόκλιση ίση με ένα. Αυτό είναι ιδιαίτερα απαραίτητο για τα SVM.

Έπειτα εφάρμοσα ανάλυση κύριων συνιστωσών (Principal Component Analysis - PCA) για τη διατήρηση του 95% της πληροφορίας (η εκφώνηση απαιτεί ρητά περισσότερο από το 90%). Η επιλογή του αριθμού των στηλών που θα κρατηθούν ώστε να εξηγούν το 95% της πληροφορίας γίνεται αυτόματα στον κώδικα. Δηλαδή δεν επιλέγουμε εμείς πόσες στήλες θα κρατήσουμε, αλλά το πρόγραμμα τις επιλέγει αυτόματα <u>ώστε</u> να διατηρείται το 95% της πληροφορίας.

Το PCA ολοκληρώθηκε σε 2.87 λεπτά (με πόρους από τη CPU) για το 95% της πληροφορίας και κράτησε 207 στήλες. Συνεπώς ο πίνακας X των features έχει 50.000 γραμμές και 207 στήλες.

Για πειραματισμό, έβαλα να κρατήσει το 60% και το 90% της πληροφορίας. Τα στατιστικά φαίνονται στον Πίνακα I.

Οι 50.000 γραμμές είναι οι 50.000 εικόνες του CIFAR-100 dataset. Οι 207 στήλες είναι τα pixels που απαιτούνται για τη διατήρηση του 95% της πληροφορίας. Δηλαδή για να κρατήσουμε το 95% της πληροφορίας (το ζουμί) δεν μας χρειάζονται όλες οι 3072 στήλες.

Πίνακας I
Χρόνος για την εφαρμογή PCA

Διακύμανση (%)	Στήλες	**Time (minutes)**	Πόροι
60.72	6	2.56	CPU
90.02	90	2.77	CPU
95.02	207	2.78	CPU

Παρατηρώ ότι ο χρόνος εφαρμογής του PCA στον πίνακα διάστασης 50.000 επί 3.072 είναι πανομοιότυπος για οποιαδήποτε διακύμανση. Αυτό εξηγείται από το γεγονός ότι η πράξη του PCA βασίζεται στον υπολογισμό όλων των ιδιοτιμών και των ιδιοδιανυσμάτων του πίνακα X. Αυτό είναι το χρονοβόρο βήμα. Έπειτα το PCA κρατάει τις συνιστώσες που εξηγούν το 95% της πληροφορίας.

Σε ότι ακολουθεί, έχουμε κρατήσει το 95% της πληροφορίας.

III . Support Vector Machine

Υλοποίησα το SVM χρησιμοποιώντας τις έτοιμες βιβλιοθήκες του sklearn.svm.

A′. *Grid Search*

Για να βρω τις βέλτιστες παραμέτρους για το SVM μου, χρησιμοποίησα Grid Search. Για να το κάνω αυτό επέλεξα ένα υποσύνολο 2.000 εικόνων από τις 50.000 με τυχαίο ανακάτεμμα (shuffle). Είναι οι 2.000 εικόνες αντιπροσωπευτικό δείγμα για να βρει τις βέλτιστες παραμέτρους για τις 50.000 εικόνες· Ελπίζω ναι, αφού τις διάλεξα τυχαία. Φυσικά εν συνεχεία θα παραθέσω πίνακες με διάφορα δείγματα.

Δημιούργησα έναν πίνακα (grid) με όλους τους συνδυασμούς για τα C και gamma. Για κάθε συνδυασμό εκπαιδεύω ένα SVM, μετράω την ακρίβειά του και κρατάω το καλύτερο.

Εκτελώ cross validation για 11 διαφορετικούς συνδυασμούς παραμέτρων σε 3 folds. Χωρίζω τις 2.000 εικόνες σε τρία ίσα μέρη.

- 1η δοκιμή: εκπαίδευσε στα μέρη A+B και έλεγξε στο Γ.
- 2η δοκιμή: εκπαίδευσε στα μέρη A+Γ και έλεγξε στο B.

- 3η δοκιμή: εκπαίδευσε στα μέρη B+Γ και έλεγξε στο A.

11 συνδυασμοί επί 3 δοκιμές = 33 εκπαιδεύσεις, δηλαδή εκπαίδευσα 33 SVM μοντέλα, μέτρησα τις ακρίβειες και κράτησα το καλύτερο. Η εκπαίδευση των 33 μοντέλων διήρκησε 30.66 δευτερόλεπτα.

Το καλύτερο αποτέλεσμα που πήρα είναι $C = 10$ και $gamma = scale$ και rbf kernel. Η καλύτερη ακρίβεια στο υποσύνολο των 2.000 εικόνων είναι 12.5%. Αυτή η ακρίβεια αναμένεται να αυξηθεί όσο προσθέτω εικόνες. Θα το δούμε στην επόμενη υποενότητα.

Το C είναι μια παράμετρος που ελέγχει την αυστηρότητα του μοντέλου. Μικρές τιμές του C του τύπου 0.1 σημαίνει ότι το μοντέλο δεν είναι τόσο αυστηρό στα λάθη και επιτρέπει εσφαλμένες ταξινομήσεις. Στη περίπτωσή μας $C = 10$ οπότε το μοντέλο προσπαθεί κατά το δυνατόν να μην κάνει λάθη. Το $gamma$ υποδηλώνει την εμβέλεια επιρροής κάθε εικόνας. Μικρή τιμή του $gamma$ σημαίνει ότι η εικόνα επηρρεάζει μόνον την πολύ κοντινή της περιοχή. Το Grid Search μας είπε να το ρυθμίζουμε αυτόματα. Επίσης μας είπε να χρησιμοποιήσουμε μη γραμμικό πυρήνα RBF για να διαχωρίσουμε τις κλάσεις (δηλαδή όχι απλώς μια ευθεία γραμμή).

B′. Τελική εκπαίδευση και αξιολόγηση

Τώρα θα εκπαιδεύσω το τελικό SVM μοντέλο χρησιμοποιώντας τις παραμέτρους που μου βρήκε το Grid Search. Θα παραθέσω τα αποτελέσματα σε πίνακα. Τα πειράματά μου θα είναι στο σύνολο των 50.000 εικόνων του train set.

Σημείωση: οι χρόνοι εκπαίδευσης και ελέγχου είναι μετρημένοι σε λεπτά.

Το train time είναι ο χρόνος που χρειάστηκε για να εκπαιδευτεί το μοντέλο και το test time είναι ο χρόνος που χρειάστηκε το μοντέλο να βγάλει την πρόβλεψη.

Πίνακας II
Ακρίβεια του μοντέλου SVM για $C = 10$, $gamma$=SCALE, KERNEL = RBF

Training Acc	Test Acc	Train Time	Test Time
96.40	29.40	8.64	4.22

Πίνακας III
Ακρίβεια του μοντέλου SVM για $C = 1$, $gamma$=SCALE, KERNEL = RBF

Training Acc	Test Acc	Train Time	Test Time
46.54	26.55	6.80	4.14

Έτρεξα το πείραμα για linear kernel. Ο κώδικας έληξε (timeout, πάνω από μία ώρα), πράγμα το οποίο σημαίνει ότι ο αλγόριθμος δεν κατάφερε να διαχωρίσει γραμμικά τα δεδομένα. Δεν πειράζει όμως, βάζουμε 10.000 εικόνες για να μην περιμένουμε για πάντα (!) και ξανά τρέχουμε το πείραμα για το γραμμικό kernel.

Πίνακας IV
Ακρίβεια του μοντέλου SVM για $C = 10$, $gamma$=AUTO, KERNEL = LINEAR

Training Acc	Test Acc	Train Time	Test Time
99.99	13.53	0.56	0.66

Γ′. Σχόλια

Παρατηρώ ότι την καλύτερη ακρίβεια την επιτυγχάνει ο συνδυασμός $C = 10$, gamma = scale και kernel = rbf. Έχουμε μεγάλη τιμή στο C, δηλαδή δεν επιτρέπουμε στο μοντέλο να κάνει λάθη. Παρ′ όλ′ αυτά, το μοντέλο παρουσιάζει overfitting, δηλαδή έμαθε απ′ έξω τα δεδομένα εκπαίδευσης. Γι′ αυτό και η τεράστια διαφορά μεταξύ train και test accuracy. Ρίχνοντας την παράμετρο C στο 1, παρατηρούμε ότι επιτρέπουμε στο μοντέλο να κάνει λάθη και έτσι έπεσε το training accuracy, οπότε η διαφορά μεταξύ train και test accuracy μειώθηκε. Παραμένει όμως χειρότερη επιλογή για την ακρίβεια από ότι το πρώτο μοντέλο. Όπως αναμέναμε, το χειρότερο μοντέλο είναι αυτό με γραμμικό πυρήνα.

Μια σημαντική παρατήρηση είναι ο υψηλός χρόνος πρόβλεψης για rbf πυρήνα. Δηλαδή πρέπει να περιμένεις κανείς 4 ολόκληρα λεπτά για να ολοκληρωθεί η πρόβλεψη στο test set. Κάτι που όπως θα δούμε δεν συναντάται στο MLP.

IV . Multilayer Perceptron με Hinge Loss

Θα υλοποιήσουμε σε PyTorch ένα πολυστρωματικό νευρωνικό δίκτυο ενός κρυφού επιπέδου που χρησιμοποιεί Hinge Loss για τη βελτιστοποίηση.

Ο μαθηματικός τύπος για το Hinge Loss είναι

$$L(\mathbf{w}, \mathbf{x}, y) = max(0, 1 - y(\mathbf{w} \cdot \mathbf{x})$$

όπου

- $\mathbf{w}$ είναι το διάνυσμα βαρών
- $\mathbf{x}$ είναι το διάνυσμα χαρακτηριστικών στην είσοδο
- y είναι η ετικέτα

Η βιβλιοθήκη PyTorch δεν δουλεύει με numpy arrays αλλά με Tensors, οπότε μετατρέπουμε τα δεδομένα μετά την εφαρμογή του PCA.

Το MLP θα έχει την εξής αρχιτεκτονική:

- Input Layer: κόμβοι όσες και οι στήλες του PCA.
- Hidden Layer: ένα κρυφό επίπεδο. Θα βάλω 256 νευρώνες. Χρησιμοποιώ τη συνάρτηση ενεργοποίησης reLU η οποία εισάγει μη γραμμικότητα.
- Output Layer: κόμβοι όσες και οι κλάσεις, δηλαδή 100.

Το loss function θα είναι το Hinge Loss και ως optimizer επιλέγω τον Adam.

Εκτελώ το βρόχο εκπαίδευσης. Κάθε 5 εποχές τυπώνω το loss, δηλαδή το σφάλμα του μοντέλου. Παρατηρώ (από την εκτέλεση του κώδικα) ότι με την πάροδο των εποχών το σφάλμα μειώνεται το οποίο σημαίνει ότι το μοντέλο μαθαίνει.

Α΄. Αποτελέσματα

Θα παραθέσω πίνακες για διάφορες τιμές των παραμέτρων εκπαίδευσης. Συγκεκριμένα, για διαφορετικά learning rates, διαφορετικό αριθμό νευρώνων στο κρυφό επίπεδο και διαφορετικές εποχές.

Το test accuracy είναι αυτό που μας νοιάζει. Αυτή είναι η τελική ακρίβεια του μοντέλου. Απλώς αναφέρουμε και το train accuracy για να διαπιστώσουμε τυχόν overfitting ή underfitting.

Σημειώνω το train time, δηλαδή το χρόνο που χρειάστηκε το μοντέλο για να εκπαιδευτεί και το test time, δηλαδή το χρόνο που χρειάστηκε το (ήδη εκπαιδευμένο) μοντέλο για να δώσει την πρόβλεψή του.

Πίνακας V
Ακρίβεια του MLP με LEARNING RATE = 0.001 και 256 νευρώνες στο κρυφό επίπεδο.

Εποχές	**Train Acc**(%)	**Test Acc**(%)	**Train Time**	**Test Time**
5	56.65	19.15	0.31	0.034
10	57.70	19.56	0.53	0.031
50	60.90	19.13	3.15	0.043
75	62.60	18.72	4.37	0.031
100	63.91	18.57	5.53	0.031

Πίνακας VI
Ακρίβεια του MLP με LEARNING RATE = 0.01 και 256 νευρώνες στο κρυφό επίπεδο.

Εποχές	**Train Acc**(%)	**Test Acc**(%)	**Train Time**	**Test Time**
5	33.77	14.05	0.23	0.053
10	34.74	14.10	0.46	0.044
15	35.42	14.74	0.69	0.033
50	37.29	13.81	2.30	0.031
75	38.69	13.99	3.47	0.031
100	39.38	13.52	4.62	0.033

Πίνακας VII
Ακρίβεια του MLP με LEARNING RATE = 0.001 και 128 νευρώνες στο κρυφό επίπεδο.

Εποχές	**Train Acc**(%)	**Test Acc**(%)	**Train Time**	**Test Time**
5	23.70	16.69	0.21	0.024
10	25.09	19.91	0.45	0.023
15	26.50	20.40	0.69	0.022
30	27.77	20.43	1.31	0.020
35	28.16	20.69	1.52	0.020
50	28.74	19.91	2.13	0.020
75	28.89	19.64	3.19	0.019
100	29.15	19.25	4.23	0.019

Επί της ουσίας πειραματίστηκα για διάφορες εποχές, πολλούς/λίγους νευρώνες και γρήγορο/αργό learning rate.

Πίνακας VIII
Ακρίβεια του MLP με LEARNING RATE = 0.01 και 128 νευρώνες στο κρυφό επίπεδο.

Εποχές	**Train Acc**(%)	**Test Acc**(%)	**Train Time**	**Test Time**
5	17.52	13.90	0.21	0.054
50	15.25	12.23	2.13	0.031
75	13.61	10.65	3.21	0.019
100	12.61	10.62	4.30	0.023

Β΄. Σχόλια

Παρατηρώ από τους πίνακες V, VI ότι για δεδομένο αριθμό νευρώνων στο κρυφό επίπεδο, η ακρίβεια πέφτει όταν μειώνεται το learning rate. Επίσης, για δεδομένο learning rate, η ακρίβεια αυξάνεται ελαφρώς στη περίπτωση των 256 νευρώνων, ενώ μειώνεται ελαφρώς στους 128.

Σημαντική σημείωση είναι ο ταχύτερος χρόνος εκπαίδευσης σε σύγκριση με τα SVM καθώς και ο ταχύτατος χρόνος πρόβλεψης!

Ακόμη, παρατηρώ ότι από τις πρώτες κιόλας εποχές βρίσκω τη μέγιστη ακρίβεια. Δεν χρειάζεται δηλαδή να φτάσω τις 50, 75 ή και τις 100 εποχές για να μεγιστοποιήσω την ακρίβειά μου.

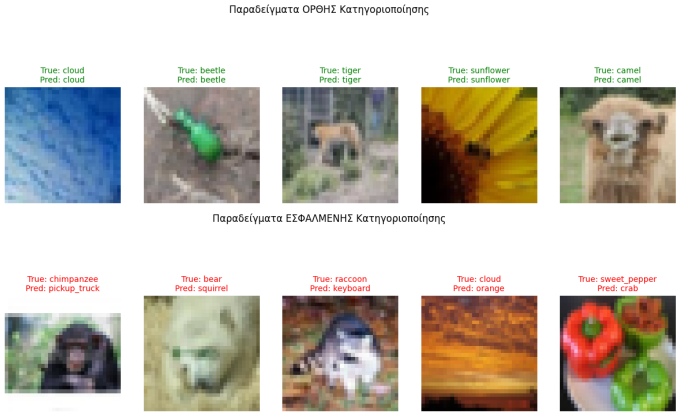
V . Σύγκριση με ταξινομητές 1-NN, 3-NN, Nearest Centroid

Όπως είδαμε στην ενδιάμεση εργασία, η ακρίβεια έκαστου ταξινομητή φαίνεται στο παρακάτω πινακάκι. Επειδή σε αυτή την εργασία χρησιμοποιήσαμε PCA, βάλαμε το πινακάκι που αφορά το PCA. Οι χρόνοι εδώ είναι σε δευτερόλεπτα.

Πίνακας IX
Ακρίβεια ταξινομητών με χρήση PCA

Ταξινομητής	**Accuracy**	**Train Time**	**Predict Time**
1-NN	19.41	0.02	6.75
3-NN	16.30	0.01	5.51
Nearest Centroid	10.03	0.08	0.02

VI . Ορθή και Εσφαλμένη Κατηγοριοποίηση



VII . Συμπέρασμα

Το SVM με $C = 10$, gamma = scale, kernel = rbf πέτυχε την καλύτερη ακρίβεια από όλα τα SVM και MLP. Αυτό εξηγείται από τον πανίσχυρο μη γραμμικό πυρήνα RBF, ο οποίος λυγίζει το χώρο για να βρει πολύπλοκα μη γραμμικά μοτίβα και επιφάνειες. Από την άλλη, το MLP έχει μόνο ένα κρυφό επίπεδο και ταυτόχρονα δεν είναι συνελικτικό, πράγμα το οποίο περιορίζει τις δυνατότητές του.

Ακόμη, ο ταξινομητής 1 - Nearest Neighbour πέτυχε ακρίβεια εφάμιλλη με το καλύτερο MLP. Αυτό δείχνει ότι το νευρωνικό δεν μπόρεσε να γενικεύσει, ίσως λόγω της ρηχής (shallow) δομής του.